

Informe del Compilador HULK

David Sánchez

June 18, 2025

Abstract

Contents

1 Introducción

1.1 Motivación

1.2 El Lenguaje HULK

1.3 Objetivos

1.4 Estructura del Informe

2 Análisis Léxico

2.1 Tokens

2.2 Expresiones Regulares

2.3 Implementación

3 Análisis Sintáctico

3.1 Gramática

3.2 Tipo de Parser

3.3 Árbol de Sintaxis Abstracta (AST)

3.4 Implementación

4 Análisis Semántico

4.1 Comprobación de Tipos

4.2 Reglas de Alcance (Scope)

4.3 Tabla de Símbolos

4.4 Manejo de Errores

5 Generación de Código

5.1 Representación Intermedia

5.2 Arquitectura de Destino

5.3 Proceso de Generación

6 Pruebas

7 Conclusiones

7.1 Resumen del Trabajo

7.2 Trabajo Futuro

References

- [1] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools*. Pearson/Addison Wesley.

A Gramática Completa

```
input -> line
      | lines_block
      | line lines_block

lines_block -> { lines }

lines -> epsilon
      | non_empty_lines

non_empty_lines -> line
                | non_empty_lines line

line -> expr ;
     | func_assign ;
     | type_node_decl

expr -> arit_op
     | bool_expr
     | while while_expr
     | string
     | id_expr
     | func_call
     | let_assign
     | if conditional
     | member_access_expr
     | expr := expr
     | expr = expr
     | expr @ expr
     | expr @@ expr
     | expr as var_id

func_assign -> function var_id ( args_list ) => expr
            | function var_id ( args_list ) : type_id => expr
            | function var_id ( args_list ) { lines }
            | function var_id ( args_list ) : type_id { lines }

args_list -> epsilon
          | id_expr
          | args_list , id_expr

id_expr -> var_id : type_id
        | var_id

let_assign -> let var_assign_list in expr
           | let var_assign_list in lines_block

var_assign_list -> id_expr = expr
                | id_expr = expr as var_id
                | id_expr = new_expr
                | var_assign_list , id_expr = expr
                | var_assign_list , id_expr = expr as var_id
                | var_assign_list , id_expr = new_expr

new_expr -> new var_id ( expr_list )

expr_list -> epsilon
```

```

| expr
| new_expr
| expr_list , expr
| expr_list , new_expr

func_call -> var_id ( expr_list )

arit_op -> number
| ( expr )
| expr + expr
| expr - expr
| expr * expr
| expr / expr
| expr ^ expr
| expr % expr
| - expr

bool_expr -> bool
| expr >= expr
| expr > expr
| expr <= expr
| expr < expr
| expr == expr
| expr != expr
| expr & expr
| expr | expr
| ! expr
| id_expr is type_id
| func_call is var_id

conditional -> ( bool_expr ) expr else expr
| ( bool_expr ) lines_block else expr
| ( bool_expr ) expr else lines_block
| ( bool_expr ) lines_block else lines_block
| ( bool_expr ) expr elif conditional
| ( bool_expr ) lines_block elif conditional

while_expr -> ( bool_expr ) lines_block
| ( bool_expr ) expr

type_node_decl -> type type_id { type_body_elements }
| type type_id inherits type_id { type_body_elements }
| type type_id inherits type_id ( args_list ) { type_body_elements }
| type type_id ( args_list ) { type_body_elements }
| type type_id ( args_list ) inherits type_id { type_body_elements }
| type type_id ( args_list ) inherits type_id ( args_list ) { type_body_elements }

type_body_elements -> epsilon
| type_body_elements attribute
| type_body_elements method

attribute -> id_expr = expr ;
| id_expr = expr as var_id ;

method -> var_id ( args_list ) => expr ;
| var_id ( args_list ) : type_id => expr ;
| var_id ( args_list ) { lines }
| var_id ( args_list ) : type_id { lines }

```

B Fragmentos de Código Fuente

Listing 1: Ejemplo de código C++

```
// Tu c d i g o a q u
```